Complexity, Arrays, Stacks and Linked Lists

Data Structures and Algorithms - Unit 1 - Linear data structures

Measuring efficiency

Big-O notation describes how running time or memory grows with input size n, ignoring constants. Common classes from fastest to slowest: O(1), O(log n), O(n), O(n log n), O(n^2), O(2^n).

```
def linear_search(a, x):        # O(n)
    for i, v in enumerate(a):
        if v == x:
            return i
    return -1
```

Arrays

- Contiguous memory; access by index is O(1).
- Insert or delete in the middle is O(n) because elements shift.
- Binary search needs a sorted array and runs in O(log n).

Linked lists

A singly linked list stores nodes with a value and a next pointer. Insertion at the head is O(1); searching is O(n). A doubly linked list adds a prev pointer, making deletion of a known node O(1).

```
class Node:
    def __init__(self, val, nxt=None):
        self.val, self.next = val, nxt

def reverse(head):
    prev = None
    while head:
        head.next, prev, head = prev, head, head.next
    return prev
```

Stack and queue

- Stack - LIFO. push, pop, peek in O(1). Used for function calls, undo, bracket matching, expression evaluation.
- Queue - FIFO. enqueue, dequeue in O(1) with a linked list or circular buffer. Used in scheduling and BFS.

> *Exam tip: when asked to evaluate a postfix expression, push operands and, on each operator, pop two values (right operand first), apply the operator and push the result.*